AMETECS INDIA PRIVATE LIMITED

DOCUMENT 2 OF 2 · TECHNICAL

Technical Logic & Specifications

Ametecs Managed Mobile Platform — architecture, data, algorithms and APIs, end to end

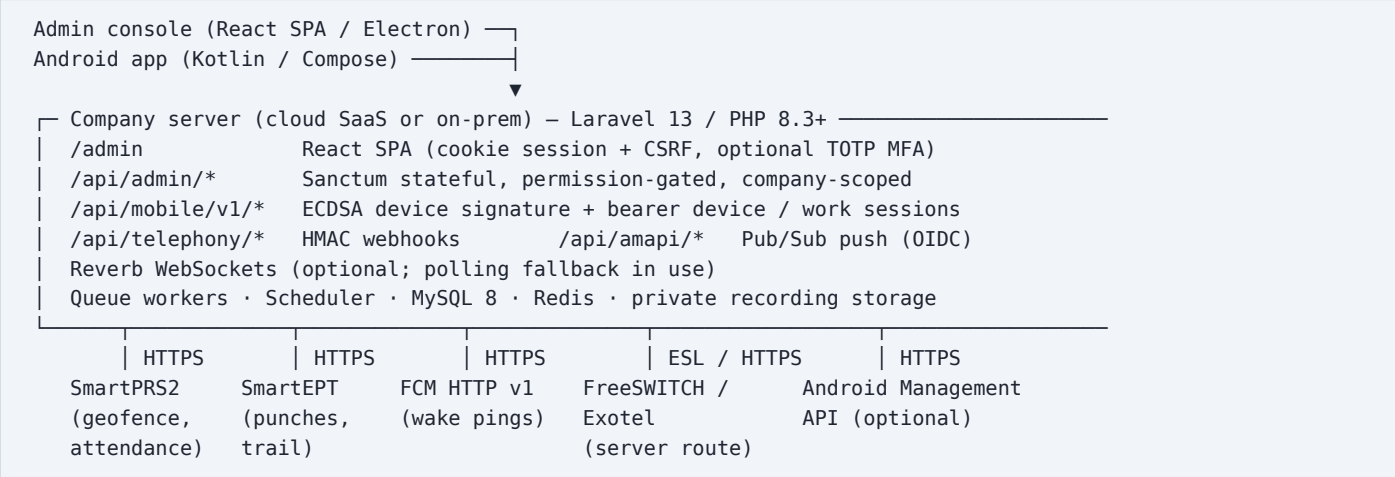
Product	Ametecs Managed Mobile Platform (working name; project "Caller" / Smart caller)
Document	Technical logic and specifications — end to end
Companion	Document 1: Business Logic Specification
Audience	Developers, tech consultant, implementation and support engineers
Version / date	v1.0 · 7 October 2026
Basis	Source code, migrations, routes and docs in C:\laragon\www\Smart caller as of 7 Oct 2026

Contents

1. Scope and system overview . . . 3
2. Multi-tenancy (company isolation) . . . 4
3. Security and authentication . . . 4
4. Data model . . . 5
5. State machines . . . 6
6. Calling engine . . . 7
7. Policy engine . . . 9
8. Mobile sync and offline model . . . 10
9. Supervision services . . . 10
10. Push notifications (FCM) . . . 11
11. Field-force integration (SmartPRS2, SmartEPT) . . . 11
12. API reference . . . 12
13. Android application . . . 13
14. Background processing . . . 14
15. Configuration . . . 14
16. Deployment . . . 15
17. Verification status . . . 15
18. Known limitations, risks and roadmap . . . 16

1. Scope and system overview

This document specifies the Ametecs Managed Mobile Platform (AMMP, project "Caller") end to end: architecture, data model, state machines, algorithms, APIs, security, integrations, jobs, configuration, deployment and verification status. It describes the code in `C:\laragon\www\Smart caller` as of 7 October 2026. Business rules are in Document 1; this document explains how they are implemented.



1.1 Technology stack

Layer	Technology	Notes
Server	Laravel 13.34, PHP 8.3–8.5, Sanctum 4.3, Reverb 1.12	Avoids 13-only APIs; also runs on Laravel 12.
Domain core	Plain PHP in `app/Domain`, no framework imports	Security-critical rules unit-tested in isolation (PHPUnit).
Database / cache	MySQL 8 (or MariaDB 10.11+), Redis 7	52 tables, 109 foreign keys (schema check).
Admin console	React + TypeScript + Vite	Builds into `backend/public/admin`; served at /admin.
Desktop	Electron shell + NSIS installer	Sandboxed, context-isolated, navigation-locked; loads the server origin only.
Android	Kotlin 2.2, Jetpack Compose, AGP 8.11, Gradle 8.14, minSdk 28 (Android 9)	Package `in.ametecs.managedmobile`; WebRTC via io.github.webrtc-sdk.
Push	Firebase Cloud Messaging HTTP v1	Data-only "sync" pings.
Media	WebRTC; coturn TURN with REST ephemeral credentials	Voice, screen and camera sessions.
Telephony (server route)	FreeSWITCH ESL adapter, Exotel adapter, simulator	Behind a `TelephonyAdapter` interface.

1.2 Repository layout

```
backend/   app/Domain      framework-free rules (Calling, Policy, Devices, Media, Security, Sync, Tenancy, Telephony…)
           app/Services    CallService, SessionService, DeviceApprovalService, PolicyService, ChatService,
                           MediaSessionService, PushService, FieldForceIntegration, ImportService, CustomerAssignmentService
           app/Http        Controllers/{Admin,Mobile,Management,Telephony}, Middleware (10)
           app/Jobs        FetchRecording, TranscodeRecording, ReconcileRecordings, RecordingWatchdog, ApplyRetention,
                           ProcessImport, SyncAmapiPolicy, SimulateCallProgress
           app/Telephony   TelephonyAdapter, FreeSwitchAdapter, ExotelAdapter, SimulatorAdapter, EslConnection
           database/       9 migrations, CompanyDefaults + DatabaseSeeder;  routes/ api.php, console.php
admin/     src/pages (Overview, Employees, Supervision, Calls, Customers, Devices, Policies, Settings, Integrations, Login)
```

```
            src/lib (api, auth, realtime, sound, unread)
android/    core/ (pure Kotlin, tested)  app/ (Compose UI, calls, data, media, policy, push, realtime, security, work)
desktop/    Electron shell        tools/  backup/restore, schema check, recording verification, Kotlin↔PHP interop
```

2. Multi-tenancy (company isolation)

- `companies` is the tenant root. Every tenant table has `company_id NOT NULL` + FK; composite unique keys include company_id.
- **Resolution order** (`Domain\Tenancy\CompanyResolver`): fixed mode (`TENANCY_MODE=fixed`) → verified custom domain → workspace slug (`X-Workspace` header or `{slug}.base-domain`). Conflict or no match → 404. No fallback. In `APP_ENV=local`, localhost requests default to `TENANCY_LOCAL_WORKSPACE`.
- `BelongsToCompany` global scope **throws** if a query runs without a bound company (fail-closed), except explicitly marked platform code (`withoutScope`).
- Queued jobs carry company_id and re-bind tenancy in `handle()`; scheduler iterates active companies via `CurrentCompany::run()`.
- Storage keys prefixed `companies/{id}/…`; WebSocket channels `private-company.{cid}.device.{did}`, `.admin`, `.conversation.{id}`, authorised against the principal's company.

3. Security and authentication

3.1 Mobile authentication layers

Layer	Mechanism	Used for
Public	Company resolution only; rate-limited (register 20/IP/hour)	bootstrap, branding/logo, devices/register
Device signature	Headers X-Installation-Id, X-Timestamp, X-Nonce (16–64 [A-Za-z0-9_-]), X-Signature = base64 DER ECDSA-P256-SHA256 over `METHOD\nPATH?QUERY\nTS\nNONCE\nhex(sha256(body))`. ±300 s skew; nonce single-use (Redis).	device/status, device/session, auth/refresh
Device session	Bearer `dst_…`, 15 min, opaque, SHA-256 hashed at rest	auth/login, policy, policy/ack, device/push-token
Work session	Bearer `wst_…`, 15 min access + 12 h refresh (refresh needs device signature). Bound to device + user + credential epoch. Every request re-checks company, approval, epoch, employee status, assignment, permission (+compliance on some routes).	Everything else

- **Device identity**: EC P-256 key pair generated in Android Keystore (non-exportable, StrongBox when available) + random 128-bit installation ID. Server stores only the public key (SPKI); a key fingerprint can exist once globally.
- **Credential epoch**: incremented on approve/revoke; all device and work sessions carry the epoch they were issued under, so a bump invalidates everything at the next request (no token caching).
- Responses to work-session requests carry `X-Offline-Lease-Expires`. Tokens never appear in URLs.
- Cross-language interop is verified: a request signed by the Kotlin client is verified by the PHP server (`tools/interop-check.sh`).

3.2 Admin authentication

- Sanctum SPA cookie session; console calls `GET /sanctum/csrf-cookie` and sends `X-XSRF-TOKEN` on mutations. No tokens in browser storage.
- TOTP MFA (RFC 6238) — `ADMIN_MFA_REQUIRED`; off by default in local, and set off by decision (no authenticator step wanted).
- 12 h absolute / 30 min idle expiry. Permission middleware `perm:<key>` on every route; supervisors scoped to supervised teams unless `scope.all_teams`.

3.3 Other controls

- Telephony webhooks: `X-Telephony-Signature = hex(HMAC_SHA256(secret, ts + "." + rawBody))`, ±300 s.
- AMAPI Pub/Sub push: OIDC token verified against `AMAPI_PUBSUB_AUDIENCE`.
- Integration API keys stored encrypted (model cast). Logs pass through a redacting tap. Security headers middleware. Spreadsheet export sanitiser (formula injection).
- Production boot guard: refuses simulator telephony, log push driver, debug mode, or non-https APP_URL.

3.4 Permission keys

`devices.view` · `devices.approve` · `devices.revoke` · `devices.wipe` · `policies.view` · `policies.edit` · `policies.publish` · `employees.view` · `employees.manage` · `customers.view` · `customers.import` · `customers.assign` · `calls.view` · `recordings.play` · `recordings.download` · `location.view` · `media.screen` · `media.screenshot` · `media.camera` · `chat.use` · `reports.export` · `branding.manage` · `settings.storage` · `settings.retention` · `telephony.configure` · `audit.view` · `system.health` · `scope.all_teams` · `mobile.work` · `calls.make`

Role templates (seeded per company by `CompanyDefaults`): company_admin (all except mobile.work/calls.make), it_admin, supervisor, employee (mobile.work, calls.make, chat.use), auditor.

4. Data model

Area (migration)	Tables
Company (000001)	companies, company_domains, company_branding, platform_admins
Users (000002)	users, permissions, roles, permission_role, role_user, teams, team_user
Devices (000003)	managed_devices, device_credentials, device_assignments, device_approval_events, device_sessions, work_sessions, push_registrations, device_commands
Policy (000004)	policy_profiles, policy_versions, policy_assignments, device_policy_snapshots, policy_acknowledgements
Customers (000005)	import_jobs, import_row_errors, customers, customer_assignments, disposition_codes
Calls (000006)	calls, call_events, recordings, call_dispositions, follow_ups
Supervision (000007)	client_events, location_events, activity_events, conversations, conversation_participants, messages, message_receipts, media_sessions, media_screenshots, audit_events, retention_jobs
Integration (2026_10_06)	integration_connections, integration_outbox

4.1 Key tables

Table	Important columns and constraints
managed_devices	asset_tag, installation_id (uuid), amapi_device_name, manufacturer/model/os/sdk/app_version, approval_state (pending|approved|rejected|revoked), approval_code (9) + expiry, management_mode (amapi|app_only), management_status, capabilities json, credential_epoch, assigned_user_id, team_id, desired/applied/last_good_policy_fingerprint, policy_state, compliant, compliance_detail json, presence, work_state (signed_out|idle|active|on_call|updating_customer), battery_pct, network_type, last_contact_at, last_ip. UNIQUE(company, installation_id), (company, approval_code), (company, asset_tag).
device_credentials	device_id, public_key_pem, fingerprint (UNIQUE global), epoch, status active|revoked, revoked_at.
work_sessions	device_id, user_id, access_hash, refresh_hash (UNIQUE), epoch, access/refresh/lease_expires_at, last_validated_at, ended_at, end_reason.
customers	customer_ref (UNIQUE per company), name, phone (E.164), alt_phone, campaign, status (new|in_progress|follow_up|closed|do_not_call), notes, follow_up_on, assigned_user_id, assigned_team_id, withdrawn_at, last_import_id.
calls	uuid (UNIQUE; also FreeSWITCH origination uuid), provider (sim|freeswitch|exotel|simulator|webhook), provider_call_id, user_id, device_id, customer_id, destination, which_number, calling_mode, agent_leg, state, hangup_cause, failure_reason, initiated/agent_answered/ringing/answered/ended_at, ring/talk/total_seconds, recording_required, recording_status, disposition_status.
call_events	call_id, provider_event_id, type, occurred_at, payload. UNIQUE(call_id, provider_event_id) → duplicate events are no-ops.
recordings	call_id (UNIQUE), status, source_path, object_key, format, mime, size_bytes, sha256, duration_seconds, attempts, last_error, legal_hold, retention_expires_at, available_at, purged_at.
call_dispositions / follow_ups	One disposition per call (code, notes, follow_up_at). follow_ups: customer, user, call, due_at, status open|done|cancelled.
client_events	UNIQUE(company_id, device_id, client_event_id) — idempotent offline sync.
media_sessions	uuid, kind (screen|camera|voice), requested_by, device_id, user_id, purpose, state, timestamps, timeout_minutes, options, end_reason.
integration_connections	system (smartprs|smartept), base_url, api_key (encrypted), enabled, enforce_geofence, last_status, last_ok_at. UNIQUE(company, system).
integration_outbox	system, kind (attendance|punch|location), payload json, status pending|sent|failed, attempts, next_attempt_at, last_error, sent_at. Index outbox_due_idx.

5. State machines

5.1 Device approval (ApprovalState)

```
pending ──approve──► approved ──revoke──► revoked   (terminal)
   └──────reject──► rejected                 (terminal)
Re-approval after reject/revoke requires a NEW registration (new key pair). permitsWork() ⇔ approved.
```

5.2 Call lifecycle (CallLifecycle — pure, idempotent)

```
initiated → agent_ringing → agent_connected → ringing → answered      (forward-only by ORDER index)
hangup/failed → terminal outcome:
  answered_at set : NORMAL_CLEARING|NORMAL_UNSPECIFIED|ORIGINATOR_CANCEL|MANAGER_REQUEST|ALLOTTED_TIMEOUT → completed, else disconnected
  failed event    : failed
  agent leg never answered (state initiated/agent_ringing) → agent_unanswered
  USER_BUSY→busy · NO_ANSWER|NO_USER_RESPONSE|ALLOTTED_TIMEOUT|PROGRESS_TIMEOUT→no_answer · CALL_REJECTED→rejected
  ORIGINATOR_CANCEL|NORMAL_CLEARING|LOSE_RACE→cancelled · other→failed
TERMINAL = agent_unanswered, completed, disconnected, busy, no_answer, rejected, failed, cancelled (events after terminal ignored)
durations: ring = (answered ?? ended) − ringing ; talk = ended − answered ; total = ended − initiated
```

5.3 Recording (RecordingState)

```
not_required (terminal)
pending → recording | processing | failed
recording → processing | failed
processing → available | failed
failed → processing            (recovery on reconciliation / manual retry)
available → deleted_by_retention
failureAction(mode, onFailure): mode=off→ignore ; optional→flag ; mandatory+terminate→terminate ; else flag
```

5.4 Media session (MediaSessionState)

```
requested —accept→ accepted —start (OS consent)→ active —end(reason)→ ended
    └decline→ declined          requested/accepted older than 90 s → expired (never executed later)
active past started_at + timeout_minutes → ended(timeout)
end reasons: employee_stopped, supervisor_stopped, timeout, revoked, connection_failed, policy_changed, os_consent_denied
```

5.5 Policy application (ComplianceEvaluator)

For each catalog key: OK ⇔ result.status = applied AND verified = true. Missing mandatory → **failed** (or **applying** while any result is pending); only optional failures → **partially_applied**; else **applied**. Optional controls reporting *requires_setup* are not failures. **compliant ⇔ no failing mandatory key.**

6. Calling engine

6.1 Authorisation algorithm (CallAuthoriser::authorise)

Inputs: effective policy values, customer row (or null for manual dial), optional client number, which (primary|alternate), context {user_id, team_ids, compliant, active_call, company timezone}, now (UTC). Output: {destination, record}. Each failed check throws a `DomainException` code returned as HTTP 422/403:

```
calling.enabled            else calling_disabled
ctx.compliant              else device_not_compliant
!ctx.active_call           else call_in_progress          (+ Redis lock "call-start:{user}" 10 s)
CallingHours.isOpen        else outside_calling_hours     (company TZ; ISO days; start>end = overnight, window
                                                           belongs to the start day; start==end = closed)
customer: assigned_user==me OR (assigned_user null AND assigned_team ∈ my teams) else customer_not_assigned
          status ∉ {do_not_call, closed}                    else customer_not_callable
          destination = phone | alt_phone (no_alternate) ; other which → invalid_number_choice
          client number given AND ≠ destination            → destination_mismatch
no customer: calling.manual_dial                          else manual_dial_disabled ; number required
prefixAllowed: ^\+[1-9]\d{7,14}$ AND starts with an allowed prefix   else destination_not_allowed
record = recording.mode ≠ off
```

6.2 SIM route (CALLING_ROUTE=sim, default)

1. App → POST /api/mobile/v1/calls {customer_id, which} (work + compliant + calls.make, throttled). Before authorising, any earlier non-terminal SIM call of this user is closed as cancelled / result_not_reported.
2. startSim creates calls (provider sim, calling_mode sim, state initiated) and a pending recordings row if required; sets device work_state = on_call; audits; broadcasts admin feed. Returns call uuid + destination.
3. App dials via TelecomManager.placeCall (ACTION_CALL; falls back to the dialer UI without CALL_PHONE). Default Phone app must be ODialer on OPPO/OnePlus/Realme (odialerSupported = installed OR manufacturer ∈ {oppo, oneplus, realme}); any default dialer elsewhere.
4. After the call, the app reads CallLog (newest outgoing entry to the last 10 digits since start; DURATION = talk) and searches MediaStore for the newest audio file since call start whose path/name looks like a call recording of that number (OEM folders: MIUI/sound_recorder/call_rec, Recordings/Call, Music/Recordings/Call Recordings, Record/Call…).
5. POST calls/{uuid}/sim-result {talk, ring, started_at, recording_found, diag}: idempotent under row lock. Start time clamped to [initiated−5 min, now]; ended = start + ring + talk (≤ now); state completed if talk>0 else no_answer; hangup_cause SIM_COMPLETED/SIM_NO_ANSWER. Recording: unanswered + nothing found → not_required; recording_found=false → failed with "No call recording found on the phone…" (+ audio folders seen) and admin alert. Device → updating_customer.
6. POST calls/{uuid}/recording (multipart, after terminal state): extension ∈ {m4a, mp4, aac, mp3, wav, ogg, opus, amr, awb, 3gp, 3gpp}, sniffed MIME audio/* or video/mp4|3gpp|octet-stream, ≤ 50 MB. Stored privately at companies/{cid}/recordings/{Y/m}/{uuid}.{ext}, size verified, SHA-256 stored, retention_expires_at = now + retention.recording_days → processing → available. Re-upload of an available recording is a no-op.
7. POST calls/{uuid}/disposition {code, notes, follow_up_at}: only after the call ends; code must be active; follow-up mandatory when the code requires it; creates call_dispositions + follow_ups.

6.3 Recording gate on Android

```
store: rec_block_since (ms), rec_ok (0/1)        applies only when company calls_recorded == true
on first run / install:   rec_block_since = now ; rec_ok = 0        → calls blocked (RecordingGateDialog)
unlock check:  SimCalls.latestCallRecordingMs(since = rec_block_since) != null  → rec_ok = 1
after each answered call: if no recording file found → rec_ok = 0 ; rec_block_since = now
dialog action: TelecomManager.ACTION_SHOW_CALL_SETTINGS (fallback: default-apps screen)
```

Rationale: Android exposes no API to read or set another app's auto-record switch, so proof-of-file in MediaStore is the only Google-policy-compliant signal. READ_CALL_LOG is a restricted permission on public Google Play — distribute as a private/managed app or sideload.

6.4 Server routes (CALLING_ROUTE=server)

- **FreeSWITCH**: bgapi originate over ESL. A-leg sofia/gateway/<gw>/<employee number> (pstn_callback) or user/dev_<device>@domain (sip_endpoint, media client not built); on answer bridge B-leg to customer with execute_on_answer=record_session::ametecs/<cid>/<Y>/<m>/<uuid>.wav. All ESL values pass strict character-class checks (OriginateCommand). php artisan telephony:listen maps ESL events (EslEventMapper) into CallService::ingest; reconnects with backoff.
- **Exotel**: POST /v1/Accounts/{sid}/Calls/connect.json (From employee, To customer, CallerId ExoPhone, Record=true, RecordingChannels=dual, signed StatusCallback). Callback body is never trusted — it only triggers GET /Calls/{sid}.json?details=true; the scheduler also syncs open Exotel calls every minute.
- **HTTP events** from another Ametecs telephony system: POST /api/telephony/v1/events with types agent_ringing, agent_answered, customer_ringing, customer_answered, customer_hangup, hangup, failed, recording_started, recording_stopped.

- **Mandatory recording watchdog**: no RECORD_START 10 s after answer → recording failed → alert → `recording.on_failure` continue_flagged or terminate (`uuid_kill`). FetchRecording retries up to 5 times; reconciliation every 10 min; manual Retry fetch.
- **Simulator** (local/testing only): last 4 digits 0001 busy, 0002 no answer, 0003 employee did not answer, else answered ~20 s.

7. Policy engine

`PolicyCatalog` is the single source of truth: each `Control(key, type, default, scope, label, enum, minSdk)`. Scope *device* = enforced by Android Device Policy via AMAPI; *app* = enforced by the employee app or the server.

Key	Type	Default	Scope
`apps.mode / apps.packages`	enum / list	allowlist / []	device
`apps.unknown_sources_allowed`	bool	false	device
`kiosk.enabled / kiosk.settings_access`	bool / enum	false / blocked	device
`device.camera / device.microphone (SDK 31+)`	enum	allowed	device
`device.adjust_volume_disabled, set_wallpaper_disabled, screen_capture_disabled`	bool	false	device
`device.usb_data (SDK 31+)`	enum	no_file_transfer	device
`device.external_storage_disabled`	bool	true	device
`device.bluetooth_config_disabled`	bool	false	device
`device.location_mode`	enum	enforced	device
`app.speaker_allowed / app.mute_allowed`	bool	true	app
`attachments.allowed_types / attachments.max_mb`	list / int	jpeg, png, pdf… / 10	app
`sharing.destinations`	list	[chat]	app
`calling.enabled / allowed_prefixes / manual_dial`	bool / list / bool	true / [+91] / false	app
`calling.hours`	object {days,start,end}	company window	app
`calling.agent_leg`	enum	pstn_callback	app
`recording.mode / on_failure / notice`	enum / enum / string	mandatory / continue_flagged / notice text	app
`location.enabled / interval_seconds / background`	bool / int / bool	true / 300 / false	app
`live_view.enabled / screenshots / quality / fps / timeout_minutes / max_concurrent`	mixed	false / false / medium / 5 / 10 / 3	app
`camera_session.enabled / timeout_minutes`	bool / int	false / 5	app
`offline.access_minutes`	int	240	app
`revocation.active_call`	enum	terminate	app
`retention.recording_days / chat_days / location_days`	int	180 / 365 / 90	app
`compliance.mandatory`	list	apps.mode, apps.packages, device.external_storage_disabled, …	—

- **Resolution** (`PolicyResolver`): merge(company ← team ← device) key-by-key; each value records its source scope. Snapshot fingerprint (16 hex) stored as desired/applied/last_good on the device.

- **Conflict detector** errors (block publish): camera_blocked_vs_sessions, mic_blocked_vs_voip, screen_capture_vs_live_view, screenshots_without_live_view, app_not_allowlisted, app_blocklisted, kiosk_requires_allowlist, empty_calling_hours, no_allowed_prefixes, background_location_without_location. Warnings (accept to publish): mic_blocked_vs_callback, recording_without_calling, location_user_can_disable.
- **Versions**: draft → published; rollback publishes last-known-good content as a new version (audited). Assignments: company (target_id 0), team, device.
- **Device sync**: `GET policy` → {fingerprint, values, controls{key:{scope, mandatory, status, reason}}, management_mode}; app applies and reports via `POST policy/ack` {fingerprint, results{key:{status, verified, detail}}} → {state, compliant}. With AMAPI, `SyncAmapiPolicy` pushes `policies/dev-{id}` translated by `AmapiPolicyTranslator`; STATUS_REPORT notifications update compliance.

8. Mobile sync and offline model

- `GET customers?since=` returns {server_time, visible_ids[], items[]}. The app deletes every cached customer not in visible_ids (withdrawal propagation).
- `POST sync {events:[{id uuid, type, occurred_at, payload}]}`, types: disposition, note, follow_up, activity, location, chat_read. Per event → accepted | duplicate | rejected(reason).
- Validation (`QueuedEvent`): UUID id; known type; ISO-8601 with zone (fractional seconds allowed); not more than 5 min in the future; **not after the offline lease expiry** (event_after_lease_expiry); payload is an object. Stored under UNIQUE(company, device, client_event_id).
- Offline lease = last successful validation + `offline.access_minutes`; the app locks protected functions at expiry.
- Android: WorkManager `SyncWorker` (periodic + syncNow); `LocationService` foreground service, balanced-power request every `location.interval_seconds`, immediate first fix, heartbeat activity event with battery and network every 60 s; server marks presence offline after 3 min without contact.

9. Supervision services

9.1 Chat

- `ChatService`: start(supervisor, employee), send(body | file, client_message_id for offline resend), page(afterId) — fetching a thread sets read_at for the reader; attachments type/size re-checked with MIME sniffing.
- Admin: `GET conversations` (with unread + last preview), `GET chat/unread`, `POST chat/read-all`; console polls via `useUnread`; chime + desktop notification (Web Audio, no asset files) + tab title count; `/chat?c=<id>` deep link.
- Every new message triggers `PushService::ping(device, "chat")`.

9.2 Media sessions (screen, camera, internet voice)

- `MediaSessionService::request`: permission media.screen / media.camera / chat.use (voice); device approved, assigned, signed in, contact < 10 min; supervisor must see the employee; policy enabled; purpose ≥ 5 chars; screen concurrency ≤ live_view.max_concurrent; one open session per device. Timeouts: screen live_view.timeout_minutes, camera camera_session.timeout_minutes, voice 60.
- Signalling over REST (`media/{uuid}/signal`, `/signals` polling — admin 700 ms, phone 500 ms; request poll 2 s) with Reverb as optional transport; ICE from `media/{uuid}/ice` with coturn `use-auth-secret` ephemeral credentials (TTL 10 min).
- Voice: admin requests mic inside the click (`getMic` diagnoses Windows-blocked / Chrome-blocked / no mic / busy), phone sends offer; admin answers sendrecv (Retry mic via replaceTrack). Android `VoiceCallService` (foreground microphone), MODE_IN_COMMUNICATION + speaker; ringing notification id 1009 with Answer/Decline, missed-call notification id 1010.

- Screen: MediaProjection with per-session OS consent (mandatory on Android 14+), foreground service + in-app banner; FLAG_SECURE content appears black.
- Revocation ends media immediately (server event) or on the next 30 s heartbeat failure.

> **Limit.** Without a TURN server, WebRTC works reliably only on the same network. Configure `TURN_URLS` / `TURN_SHARED_SECRET` for mobile data.

10. Push notifications (FCM)

- Server: `PushService` (FCM HTTP v1; OAuth access token cached 50 min) sends data-only `{type:"sync", reason}` — no customer data or text. Registrations in `push_registrations` (status active|invalid|disabled).
- Phone: `AppGraph.registerPush()` → PUT `/api/mobile/v1/device/push-token` after sign-in. `push/PushService.kt`: reason media → refresh → ring; reason chat → if app open refresh UI, else fetch newest message and show notification (8 s budget). Dedupe key `chat_notified`.
- Notification actions (`NotificationActions` receiver): DECLINE, REPLY (RemoteInput → POST message), MARK_READ. Channels: `supervisor_messages`, `incoming_calls` (insistent ringtone).
- Gradle applies google-services only when `android/app/google-services.json` exists. Test: `php artisan ammp:push-test EMP001 [--reason=chat|media]`.

11. Field-force integration (SmartPRS2, SmartEPT)

11.1 Sequence

```
Phone sign-in (GPS fix) ─► Caller SessionService::login
   password ok → FieldForceIntegration::geofenceGate(user, lat, lng)
        POST {PRS}/api/v1/fieldforce/geofence/check {emp_code, lat, lng}   (X-Api-Key, scope ingest)
        ← {allowed, status, distance_km, limit_km, fence{lat,lng,radius_km,outside,limit_km}, message}
        not allowed AND enforce_geofence → audit + DomainException outside_geofence (422)
        PRS unreachable → status "unverified" (fail-open); fence cached ~14 h per user
   recordSession(login) → integration_outbox:
        smartprs attendance → POST {PRS}/api/v1/fieldforce/attendance {events:[{external_id, emp_code, type, at, lat, lng, accuracy_m}]}
        smartept punch      → POST {EPT}/api/v1/attendance/punches   (+lat, lng, accuracy_m, geo_status)
SyncController location events → recordLocation → POST {EPT}/api/v1/fieldforce/locations
        geo_status = haversine(fix, cached fence) > limit_km ? outside : within   (no-rule if no fence)
Flush: right after the response + scheduler every minute (integrations:flush)
        2xx → sent ; 422 → failed (rejected) ; other → retry backoff 1,2,4…60 min, indefinitely
```

Receiver	Endpoint	Result codes
SmartPRS2	POST /api/v1/fieldforce/geofence/check	status: within | outside | no-gps | no-rule | unknown-employee
SmartPRS2	POST /api/v1/fieldforce/attendance	per event: accepted | duplicate | unknown_employee | rejected (+ geo_status). Writes attendance_logs source "caller", device_sn CALLER; unknown codes held in attendance_pending for mapping.
SmartEPT	POST /api/v1/attendance/punches (existing, extended)	Geo fields stored in biometric_logs.metadata.
SmartEPT	POST /api/v1/fieldforce/locations (ingest)	Stores field_force_locations; first outside fix → employee_compliance_events GEOFENCE_EXIT (DEVICE/MEDIUM); first inside fix sets resolved_at.
SmartEPT	GET /api/v1/fieldforce/locations?employee_code=&date= (read)	Day's trail.

- Tenant is always taken from the receiver's API key, never from the body. Every event carries `external_id` → idempotent.

- Admin endpoints: GET /api/admin/integrations (system.health), PUT integrations/{smartprs|smartept} (telephony.configure), POST …/test, POST integrations/flush. IntegrationConnection::url() strips a trailing /api or /api/vN.
- MySQL 8 reserves the word system: query with select('system','status'), never inside selectRaw.

12. API reference

12.1 Mobile — /api/mobile/v1

Method & path	Auth	Purpose
GET bootstrap · GET branding/logo	public	API version, product, company, logo URL + SHA-256
POST devices/register	public	{installation_id, public_key, device info} → {state, approval_code, poll_after_seconds}; different key for same installation → installation_conflict
GET device/status · POST device/session	signed	Approval polling; device token (approved only)
POST auth/login	device	{employee_code, password, lat?, lng?, accuracy?} → tokens, lease, employee
POST auth/refresh · POST auth/logout	signed / work	Rotate tokens; end work session only
GET me	work	Employee, compliance, lease, realtime channel, monitoring disclosure
GET policy · POST policy/ack · PUT device/push-token	device	Policy sync and FCM token
GET customers?since= · GET dispositions	work + compliant	Assigned customers; active codes
POST calls	work + compliant + calls.make	{customer_id, which, number?} → {call_id, state, destination…}
GET calls/{uuid} · POST hangup · sim-result · recording · disposition	work	Own calls only
POST sync	work	Offline event batch
GET/POST conversations…/messages · attachment	work + chat.use	Chat, receipts, attachments
GET media/pending · POST media/{uuid}/{accept|decline|start|end|signal|protected-content} · GET ice, signals	work	Employee side of sessions

12.2 Admin — /api/admin (cookie session + CSRF, permission-gated)

Group	Endpoints
Auth	POST auth/login, auth/mfa/setup, auth/mfa/verify, auth/logout; GET auth/me
Overview	GET overview
People	GET/POST employees, PATCH employees/{id}, GET/POST teams, PUT teams/{id}/members, GET roles
Devices	GET devices, devices/{id}; POST devices/{id}/approve|reject|reassign|revoke|wipe; POST enrolment-tokens; GET devices/{id}/effective-policy
Policies	GET policy/catalog; GET/POST policies; POST policies/{id}/versions, …/versions/{v}/publish {accept_warnings}, policies/{id}/rollback; PUT policy-assignments
Customers	GET imports/template; GET/POST imports; POST imports/{id}/preview|validate|commit; GET imports/{id}, imports/{id}/errors.csv; GET customers; POST customers/assign|withdraw

Group	Endpoints
Calls	GET calls, calls/{id}; POST calls/{id}/recording/playback|download|retry; DELETE calls/{id}; GET follow-ups; GET/PUT disposition-codes
Supervision	GET presence, locations; GET/POST media-sessions; POST media-sessions/{uuid}/end|signal|screenshots; GET …/ice, …/signals; GET/POST conversations, …/messages; GET chat/unread; POST chat/read-all
Settings	GET/POST branding; GET storage/estimate; GET/POST retention-jobs, POST retention-jobs/{id}/approve (different admin); GET audit, health; GET reports/calls.csv
Integrations	GET integrations; PUT integrations/{system}; POST integrations/{system}/test; POST integrations/flush

12.3 Other endpoints

GET /api/admin/recordings/{id}/stream?expires&signature (5-min signed stream) · POST /api/broadcasting/auth · POST /api/telephony/v1/events (HMAC) · POST /api/telephony/v1/exotel/{uuid} · POST /api/amapi/v1/notifications (OIDC) · GET /up. Errors: {"error": code, "message": text} — 401 auth/signature, 403 device_not_approved|device_revoked|not_assigned|forbidden|device_not_compliant|mfa_required, 404 company_unresolved|company_conflict, 422 validation/domain rule.

13. Android application

Package / file	Responsibility
AmmpApp.kt	AppGraph (DI), notification channels, push registration
MainActivity.kt	Compose host; intent handling (ANSWER, OPEN_CHAT); mic consent launcher
security/DeviceKeys.kt, SecureStore.kt	Keystore EC P-256 key, request signing; encrypted token store
data/ApiClient, ServerConfig, SessionManager, WorkRepository	HTTP + signing headers; server URL rules (locked after device session); login/refresh; offline queue and customer cache
calls/SimCalls.kt	Dialer checks (ODialer/default), placeCall, inCall, CallLog lookup, MediaStore recording search, latestCallRecordingMs, openCallSettings
calls/InCall.kt, CallRecorder.kt	Call screen/flow, result + upload; legacy in-app recorder (unused since phone-recorder decision)
policy/PolicyEnforcer.kt	Apply app-scope controls, observe device controls, build ack results honestly
work/LocationService.kt, SyncWorker.kt	Foreground location + heartbeat; periodic and immediate sync; currentFix() for sign-in
media/MediaCoordinator, WebRtcSession, MediaServices	Session polling, ringing/missed notifications, WebRTC peer, voice/screen/camera foreground services
push/PushService.kt, NotificationActions.kt	FCM receiver; Reply / Mark read / Decline actions
realtime/PusherClient.kt	Reverb (Pusher protocol) client with reconnect; polling fallback
ui/AppViewModel.kt, Screens.kt, Design.kt	Connect → wait → sign-in → policy → dashboard, customers, call outcome, chat, monitoring disclosure, recording gate dialog

Permissions used: CALL_PHONE, READ_PHONE_STATE, READ_CALL_LOG, READ_MEDIA_AUDIO / storage, ACCESS_FINE/COARSE_LOCATION (+ background via policy only), POST_NOTIFICATIONS, RECORD_AUDIO (voice calls), FOREGROUND_SERVICE_* (location, microphone, mediaProjection, camera).

Build note. The bridge cannot stage files deeper than 7 levels, so Android sources are mirrored to `Smart caller\_src` by a dev-only `copy {}` block in `android/app/build.gradle.kts`. **Delete it before release.**

14. Background processing

Job / schedule	Frequency	Purpose
`recordings:reconcile → ReconcileRecordings`	every 10 min	Flag recordings missing beyond 30 min as failed; recover found files
`media:expire`	every minute	Expire unanswered/timed-out media sessions
`exotel:sync`	every minute	Pull results/recordings of open Exotel calls (last 3 h)
`presence:sweep`	every minute	online → offline after 3 min without contact
`integrations:flush`	every minute	Retry SmartPRS2/SmartEPT outbox
`FetchRecording / TranscodeRecording`	queued	Copy from telephony, SHA-256, optional ffmpeg MP3, mark available
`RecordingWatchdog`	queued (+10 s after answer)	Mandatory recording start check
`ApplyRetention`	on approved retention job	Delete expired, non-held recordings (≤1000 per scope)
`ProcessImport`	queued	Validate/commit customer imports
`SyncAmapiPolicy`	queued	Push per-device AMAPI policy

Storage estimate (RetentionPlanner): bytes = agents × calls/day × avg talk min × 60 × (32 kbit/s ÷ 8) × retention days. Example: 50 agents × 80 calls × 3 min × 180 days ≈ 518 GB.

15. Configuration

Variable	Values / default	Purpose
`APP_PRODUCT_NAME`	Ametecs Managed Mobile	White-label name
`TENANCY_MODE / TENANCY_FIXED_COMPANY_ID / TENANCY_BASE_DOMAIN`	shared | fixed	SaaS vs on-prem
`CALLING_ROUTE`	sim (default) | server	Calling route
`SIM_RECORDER`	phone (default) | app	Who records SIM calls
`TELEPHONY_DRIVER`	freeswitch | exotel | simulator	Server-route adapter
`FREESWITCH_ESL_HOST/PORT/PASSWORD, _GATEWAY, _CALLER_ID, _RECORDINGS_BASE, _RECORDING_SOURCE`	127.0.0.1:8021, local|http	FreeSWITCH
`EXOTEL_SID/API_KEY/API_TOKEN/CALLER_ID/SUBDOMAIN`	api.in.exotel.com	Exotel
`TELEPHONY_WEBHOOK_SECRET`	—	HMAC for telephony events
`MANAGEMENT_DRIVER, AMAPI_*`	app_only | amapi	Android Management
`RECORDINGS_DISK, RECORDINGS_TRANSCODE`	recordings, mp3|none	Storage and transcode
`PUSH_DRIVER, FCM_PROJECT_ID, FCM_SERVICE_ACCOUNT_JSON_PATH`	log | fcm	Push
`TURN_URLS, TURN_SHARED_SECRET, STUN_URLS`	—	WebRTC
`ADMIN_MFA_REQUIRED`	false in local	Authenticator step

Variable	Values / default	Purpose
SANCTUM_STATEFUL_DOMAINS	all company hosts	Cookie session domains

Fixed values in config/ammp.php: device token 900 s; work access 900 s; refresh 43200 s; admin absolute 12 h; WS revalidate 60 s; rate limits register 20/IP/h, login 5/device/min, status poll 12/device/min; import ≤ 20 MB; logo ≤ 512 KB; playback URL 300 s; missing recording alert 30 min.

16. Deployment

16.1 Production topology

```
Internet — nginx (TLS, HSTS) ─┬─ php-fpm (Laravel) ── MySQL 8 (primary + encrypted off-site backups)
                              ├─ /app/* → Reverb :8080 (WebSockets)      ├─ Redis (cache, queues, sessions, nonces)
                              └─ /admin → static console                  ├─ queue workers (Supervisor) default, recordings
scheduler: * * * * * php artisan schedule:run                             ├─ telephony:listen (per FreeSWITCH node, server route)
recordings: private disk or S3-compatible (private, SSE, versioned)       └─ coturn (TURN TLS 5349 / TCP 443)
```

- **Cloud SaaS**: wildcard DNS + certificate for *.base-domain; custom domains verified by DNS TXT; provision with php artisan ammp:create-company <slug> "<name>" <admin email> --timezone=Asia/Kolkata --country=IN.
- **On-prem**: TENANCY_MODE=fixed; publicly trusted certificate (or customer-specific APK pinning the company CA); outbound internet still needed for FCM / AMAPI.
- **Backup**: tools/backup.sh nightly (single-transaction dump + recordings tar + SHA-256 manifest); tools/restore.sh refuses mismatched manifests; verify-recordings.php checks prefix, presence, size, hash. Quarterly restore drill.

16.2 Development environment (current)

```
backend:   cd "C:\laragon\www\Smart caller\backend"
           php artisan serve --host=0.0.0.0 --port=8000      (single-threaded: planned move to Laragon Apache)
           php artisan schedule:work                          (integration retries, sweeps)
admin:     cd admin && npm run build  → open http://localhost:8000/admin  (mic needs localhost/HTTPS)
android:   Android Studio → Run ▶ ; phone server URL http://<PC Wi-Fi IPv4>:8000 ; emulator http://10.0.2.2:8000
integrations (dev): PRS http://smartprs2.test/api/v1 · EPT http://192.168.0.100:8002/api/v1
```

17. Verification status

Suite	Result
Backend domain core (tenancy, signatures, TOTP, policy, conflicts, compliance, AMAPI mapping, import, authoriser, lifecycle, recording, lease, media, retention, ESL)	40 tests, 402 assertions — pass
Laravel feature/acceptance (SQLite)	54 tests — pass, except 2 pre-existing (Exotel recording; loopback workspace)
FieldForceIntegrationTest (Caller)	3 tests — pass
SmartPRS2 CallerFieldForceTest	4 tests / 30 assertions — pass (new hold-and-map case not yet run)
SmartEPT CallerFieldForceTest	2 tests — pass
Schema DDL on MariaDB 10.11	52 tables, 109 FKs, clean rollback
Backup → restore → verify; Kotlin↔PHP signature interop	Pass
Admin console (typecheck, tests, build)	Pass (dashboard mock assertion pre-existing failure)

Suite	Result
Android debug APK	Built 1 Oct; not recompiled since any-dialer, strict recording gate, voice call, notifications
Live end-to-end on Laragon (simulator)	Pass 1 Oct (register → approve → login → policy → call → disposition → revoke → 401)
Physical-device matrix, 21 manual integration cases	Not run

18. Known limitations, risks and roadmap

#	Item	Impact / next step
1	Recording depends on the OEM dialer auto-record and its file location (Pixel has none).	Validate per model; gate blocks calling if no file. Consider server route for brands without recorder.
2	READ_CALL_LOG is restricted on public Play.	Distribute privately (managed Play / sideload).
3	Uninstall / settings lock-down needs Android Enterprise device owner (AMAPI) with factory-reset enrolment; Google eligibility (commercial EMM, 500-device justification).	Build "Enrol phone (QR)" screen; confirm eligibility.
4	php artisan serve is single-threaded ("Failed to fetch", slow calls).	Move to Laragon Apache / nginx + php-fpm.
5	No TURN server.	Deploy coturn for internet calls on mobile data.
6	Reverb not running.	Polling fallback works; start Reverb in production for lower latency.
7	Double attendance if SmartEPT also pushes to SmartPRS2.	Choose one path per company.
8	Geofence verdict on GPS fixes uses fence cached at sign-in (~14 h).	Rule changes apply from next sign-in.
9	PRS geofence logic duplicated (FieldForceGeofence vs AttendanceReportController).	Merge when that controller is next edited.
10	sip_endpoint in-app VoIP agent leg not implemented; FreeSWITCH outcome variables unconfirmed.	Only if server route is revived.
11	No web console for platform administrator; no SmartEPT trail map; no integrations.manage permission.	Backlog.
12	Delete the dev-only _src copy block in build.gradle.kts before release; sign release APK and Windows installer.	Release checklist.